Automated Container deployment and Administration in the cloud

Name: Dhananjay Pramod Nidhonkar (20085410)

Module Title: Network Systems and Administration

Module Code: B9IS121

Module Instructor: Kingsley Ibomo

Word Count: 2119 words

Video URL:
https://drive.google.com/file/d/16oSySvvr2lyOplUpDx214W6P6wJ9ThcF/view?usp=sharing

Summary

This report includes of the design, implementation and evaluation of automated cloud infrastructure project for the CA1. The below project demonstrates the full lifecycle of a containerized web application on Amazon Web Servers (AWS), using Morden DevOps tools chain as Terraform, Ansible, Docker and GitHub Actions.

The infrastructure is provisioned using Terraform as Infrastructure as code (IAS), it creates the Virtual Private Cloud (VPC) and EC2 compute instance running on Amazon Linux, Identity and access management (IAM) roles, and supporting S3 storage. Configuration management and application deployment are handled by Ansible, Were in Ansible connects to EC2 instance via SSM managed by AWS this eliminating the need for open port SSH or access keys.

The Web application is containerised and packed as Docker on ngnix:alpine base, exposing the index.html file which developer created. The entire life cycle is achieved by GitHub Actions, which is triggered when there are any changes made in app/ folder were our web application is present.

Here key security outcomes include of no SSH port, no static/hardcoded AWS credentials stored in GitHub we used OIDC federation for authentication to AWS infra. This design decisions aligns with AWS architected framework security principle (AWS).

The report concludes that chosen toolchain represents industry-best-practice approach to cloud Automation.

Contents

Summary 2
1. Introduction 4
2. Infrastructure 4
2.1 Architecture 4
2.2 Prerequisites 5
3. Part A Terraform 5
3.1 Terraform- Infrastructure as code 5
4. Part B Ansible 5
5. Part C Docker Container Deployment 6
6. Part D CI/CD GitHub Actions 6
6.1 WorkFlow 1 – Infra_deploy.yml 6
Job 1: Terraform (Terraform Apply) 6
Job 2:- Ansible (depends on terraform job 1) 6
6.2 Workflow 2- app-deploy.yml (Application changes only) 7
Job: app-deploy 7
6.3 Workflow 3 Terraform Destroy 7
7. Security Design 7
8. Conclusion 8
9. References 9
10. Appendices 10
Appendix A: Architecture 10
Appendix B – CI/CD Pipeline Workflow Summary 10
Appendix C – Failure but succeed 10

1. Introduction

The emergence of cloud computing has fundamentally altered how organisations provision, configure, and maintain IT infrastructure. Manual server configuration — once the norm — is increasingly regarded as error-prone, inconsistent, and incompatible with the pace of modern software delivery (Kim G, 2021). Automation, encapsulated in the disciplines of Infrastructure-as-Code (IaC) and DevOps, has become a critical capability for organisations seeking agility, repeatability, and security in their cloud environments.

This report shows the design and implementation of automated container deployment system on AWS. Later mentioned workflows demonstrate the deployment of IAAS, PAAS, SAAS.

In report there are 3 workflows with follows the deployment of infrastructure, WEB application deployment, Web application changes, Infrastructure destroy. Certain changes need to be made at bootstrap level.

This full lifecycle of the web application uses below four tools

Tools Used	Description/Roles
Terraform	Infrastructure-as-Code – provisions VPC, EC2, IAM, S3, Security Groups
Ansible	Configuration management – installs Docker, builds and deploys the container
Docker	Containerisation – packages the Nginx web application into a portable image
GitHub Actions	CI/CD pipeline – orchestrates Terraform + Ansible on every `git push`

Table 1: Tools Used

2. Infrastructure

2.1 Architecture

The system Architure follows the layerd model were each tool address a unquie functions which are coordinated by CI/CD pipeline.Figure 2 (See Appendix A) present a diagram of the full architecture.

GitHub Actions and comprises two workflows: infra_deploy.yml, which triggers on changes to Terraform or Ansible files, and app-deploy.yml, which triggers on changes to the application source. Both workflows authenticate to AWS using OIDC federation, ensuring no long-lived credentials are stored in GitHub.

We also have Destroy workflow which is used to destroy the whole AWS infrastructure, but it is only when user confirm in GitHub Actions. Then GitHub pipeline uses terraform destroy the whole infrastructure.

2.2 Prerequisites

Local Machine Setup

Requirement	Version	Purpose
Terraform	> 1.6.0	Required for initiating OIDC, GitHub Action IAM role
AWS CLI	V2	Initialize S3 bucket
AWS Credentials	Admin or IAM user	Required for initiate start, rest pipeline uses

Table 2: Local Machine Setup Bootstrap

GitHub Repository Secret / variables

AWS-ROLE_ARN = Stores the arn value required for OIDC

S3_BUCKET_NAME = Stores the S3 bucket name onetime only

INSTANCE_ID = Stores the instance ID name onetime only

3. Part A Terraform

3.1 Terraform- Infrastructure as code

Terraform was developed by Hashicorp which an open source IAC tool that enable infrastructure to be defined in configuration language and provisioned through API call to cloud providers (HashiCorp). In this project it starts with terraform deployment here we written codes for each Job and activity which needs to be performed. This uses the JSON/YAML syntax (AWS, a).

As in this setup with networking, security groups, EC2, IAM, OIDC, S3 bucket, main.tf, ssm enabling.

4. Part B Ansible

Ansible, maintained by RedHat which is an agentless configuration management tool and automation platform which uses SSH to manage the infrastructure as in EC2 instance or buckets. (Red Hat,).

This Job 2 were ansible connects to EC2 instance using SSM manager. Benefit of using SSM no port 22 needed to be opened. No SSH key pair. Access is governed by IAM policies

In in inventory/aws.ec2.yml here we plugin queries for the AWS EC2 API for the for instance which matches the provided instance.

When instance is identified we start with main playbook file (site.yml). This file holds all the configuration and connections methods to connected and transfer the docker file to S3 bucket

and then it creates base image which is nginx:alpine server (A web server to host web application).

5. Part C Docker Container Deployment

Docker is the standard containerisation platform, which enables application and other application to use it portably. In this project we use Docker with this cmd to install our file to docker nginx:alpine container which run our web application.

```
    cd /opt/app
    DOCKER_BUILDKIT=0 docker build --pull=false -t ca1-webapp:latest .

# 7. Run container
- name: Deploy container
  ansible.builtin.raw: |
    docker rm -f webapp 2>/dev/null || true
    docker run -d \
      --name webapp \
      --restart always \
      -p 80:80 \
      ca1-webapp:latest
```

Figure 1: Docker run cmd

Web Application

The “app/index.html” file is a self-contained single web application within CSS which copied to nginx document root inside the container.

6. Part D CI/CD GitHub Actions

GitHub Actions is cloud-native CI/CD platform integrated directly into GitHub repositories. Which enables the workflows to be triggered by push, pull events(GitHub). In this project we deploy entire lifecycle by calling terraform and ansible in sequence and handling with OIDC-based Authentication to AWS.

6.1 Workflow 1 – Infra_deploy.yml

This triggers the main file which is under the terraform/ and ansible or workflows directory.

Job 1: Terraform (Terraform Apply)

1. Configure/Authenticate AWS credentials via OIDC
2. Ensure the terraform State S3 bucket exists
3. Terraform init
4. Terraform plan
5. Terraform apply
6. Capture and export instance_id, S3_bucket, public_ip

Job 2:- Ansible (depends on terraform job 1)

1. Configure/Authenticate AWS credentials via OIDC
2. Install AWS SSM Session manager plugin

3. Pip install ansible boto3 botocore (Used for aws cli)
4. Ansible-galaxy collection install amazon.aws community.aws
5. Wait for SSM agent to online (Check via ping cmd)
6. Verify SSM accepts the commands
7. Initializing the ansible playbook/site.yml
8. Shows the live output of Web Application

6.2 Workflow 2- app-deploy.yml (Application changes only)

The is Application files if user makes any changes in app/ folder then it get triggers

Job: app-deploy

1. It authenticates with AWS Credentials via OIDC
2. Once it checks the GitHub Secrets for the saved credentials.
3. Uses SSM Manager to connect to EC2 and it replace the previous code and updates the lates code via bash.
4. Prints the live application URL

6.3 Workflow 3 Terraform Destroy

The command is initiated via pipeline after the user confirm of “DESTORY”

This removes everything which have configure with keeps just the OIDC configuration and tfstate bucket as in to keep to re-deploy the architecture if need.

7. Security Design

This Project security design chosen as per AWS well-architecture framework security pillar, which identifies the loops and leaks in the AWS environment (AWS). The Following measures address each relevant domain

Security Concern	Mitigation	Framework Alignment
SSH exposure	Port 22 closed; all access via SSM	Infrastructure Protection
Terraform state exposure	OIDC federation; temporary 15-min tokens per pipeline run	Identity & Access Management
SSM relay data exposure	Private S3 bucket; 1-day object expiry lifecycle rule	Data Protection

Accidental infra destruction	prevent_destroy on OIDC provider and IAM role	Resilience
Overly broad network access	Security Group: TCP 80 and 443 inbound only	Infrastructure Protection

Table 3: Security Design Summary

Separating these concerns reduces unnecessary pipeline execution time and follows the principle of pipeline right-sizing advocated by Humble and Farley (2010).

As SSM Manager API eliminates the requirement of the Remote shell access of EC2 instance via port . where in public in case EC2 instance if misconfigured and can be accessed via public attacker easily.

As we are aware ongoing attacks on Web surface in this project we used OIDC to securely connect to AWS. OIDC is advancement of O.Auth 2.0 which short lived token and it get expires as soon our work is done(Example: - used for SSO).

8. Conclusion

The Project has demonstrated that fully automated, secured cloud-native deployment pipeline. This can be create using open-source tool such as GitHub, Terraform, Ansible with combination of AWS managed services.

We have used SSM-based connection here which is more complex to configure than ssh but also secured in terms of security. Were we don't need the port no 22 which removes the network-based attack surfaces.

We have also used OIDC based authentication in CI/CD pipelines which add a top on layer for security over standard credential storage. This can be adopted as standard practice.

The security design aligns with AWS Well-Architected Framework principles, and the toolchain represents current industry best practice for cloud-native application delivery.

9. References

Video URL:
https://drive.google.com/file/d/16oSySvvr2lyOplUpDx214W6P6wJ9ThcF/view?usp=sharing

Dhananjay GitHub, Available at: https://github.com/djdhananjay1/CA1-Networking.git [Accessed on 26 February 2026]

AWS (2024) AWS Well-Architected Framework – Security Pillar. Amazon Web Services. Available at: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html [Accessed: 20 February 2026].

AWS, AWS CloudFormation User Guide. Amazon Web Services. Available at: https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html [Accessed: 20 February 2026].

Bryant Bost, (2020) 'Create a CI/CD pipeline for Amazon ECS with GitHub Actions and AWS CodeBuild Tests', AWS blogs, 10 Feb. Available at: https://aws.amazon.com/blogs/containers/create-a-ci-cd-pipeline-for-amazon-ecs-with-github-actions-and-aws-codebuild-tests/ [Accessed: 21 Feb 2026]

AWS, AWS Systems Manager Session Manager. Amazon Web Services. Available at: https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html [Accessed: 20 February 2026].

Docker, Docker Documentation. Docker Inc. Available at: https://docs.docker.com [Accessed: 20 February 2026].

GitHub, GitHub Actions Documentation – Security hardening with OpenID Connect. GitHub. Available at: https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/about-security-hardening-with-openid-connect [Accessed: 20 February 2026].

HashiCorp, Terraform Documentation. HashiCorp. Available at: https://developer.hashicorp.com/terraform/docs [Accessed: 20 February 2026].

Humble, J. and Farley, D. (2010) Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation. Addison-Wesley Professional.

Kim, G., Debois, Willis, J. and Humble, J. (2021) The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations. IT Revolution Press.

Merkel, D. (2014) 'Docker: Lightweight Linux Containers for Consistent Development and Deployment', Linux Journal, 2014(239). Available at: https://dl.acm.org/doi/10.5555/2600239.2600241 (Accessed: 20 February 2026).

Morris, K. (2016) Infrastructure as Code: Managing Servers in the Cloud. O'Reilly Media.

OWASP (2021) OWASP Top Ten – A07: Identification and Authentication Failures. Open Web Application Security Project. Available at: https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/ (Accessed: 20 February 2026).

Red Hat Ansible Documentation. Red Hat Inc. Available at: https://docs.ansible.com (Accessed: 20 February 2026).

10. Appendices

Appendix A: Architecture

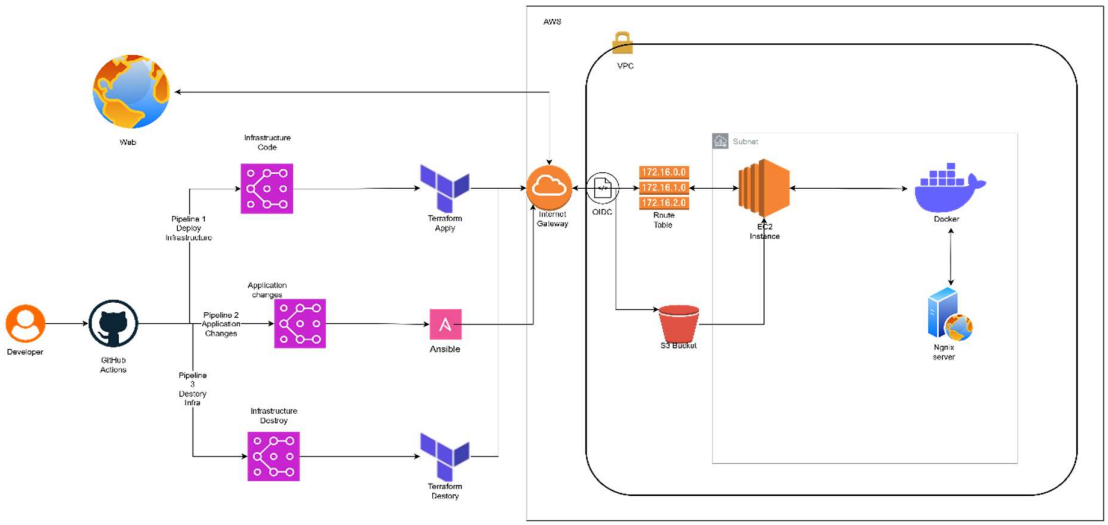

Figure 2: Project Architecture

Appendix B – CI/CD Pipeline Workflow Summary/Readme file

Pipeline	Trigger	Jobs
infra_deploy.yml	Push to terraform/**, ansible/**	Job 1: Terraform Apply Job 2: Ansible Configure (GitHub URL)
app-deploy.yml	Push to app/**	S3 Sync + SSM Run-Command
Infra_destory.yml	Enter Keyword DESTORY in GitHub	Terraform destroy

Table B1: CI/CD Pipeline summary ((GitHub URL))

Appendix C – Failure but succeed

In this project I failed multiple times as in I faced difficulty in OIDC, SSM, and SSM manager not started on ec2. I started with OIDC but at 1st time it was a success I was happy. But When I hit manually terraform destroy and then retied the same terraform apply pipeline but it failed I troubleshooted the actions and then checked in role even checked IAM roles. And I check logs of terraform destroy where I find the exact issue that terraform destroy was deleting everything it is creating and that was a issue for so in code file I mentioned to keep OIDC as prevent deletion true.

Another area I faced issue was in SSM manage from JOB2 where I waited for SSM manager to start but It was taking time more that 2 min sometime which check in logs of ansible and after identifying the issue I addressed it and asked it to check SSM manager is installed or not then check for if the service is started on the instance of not.

These 2 issues were critical and important which I addressed and resolved promptly.